



Lecture-9

Breadth-first search

(Uninformed Searching)

Course Teacher

Prof. Dr. Md. Ashikur Rahman Khan

Searching Strategies



There are **several general-purpose search algorithms** that can be used to solve these problems.

(i) Uninformed strategy

(also called blind) search strategy/algorithms

—algorithms that are given no information about the problem other than its definition.

Although some of these algorithms can solve any solvable problem, none of them can do so efficiently.

(ii) Informed strategy

(also called Heuristic/Intelligent) search algorithms,

on the other hand, can do quite well given some guidance on where to look for solutions.

Uninformed Search Strategies

- **Uninformed search** means that the strategies have no additional information about states beyond that provided in the problem definition.
- **Uninformed search strategies (blind, exhaustive, brute force)** do not guide the search with any additional information about the problem.
- All they can do is generate successors and distinguish a goal state from a non-goal state. All search strategies are distinguished by the order in which nodes are expanded.
- The uninformed search algorithms which looked through search space for all possible solutions of the problem without having any additional knowledge about search space.

Uninformed search strategies

- **Uninformed**: While searching you have no clue whether one non-goal state is better than any other. Your search is blind. You don't know if your current exploration is likely to be fruitful.
- The uninformed search algorithms which looked through search space for all possible solutions of the problem without having any additional knowledge about search space.

Informed Search Strategies

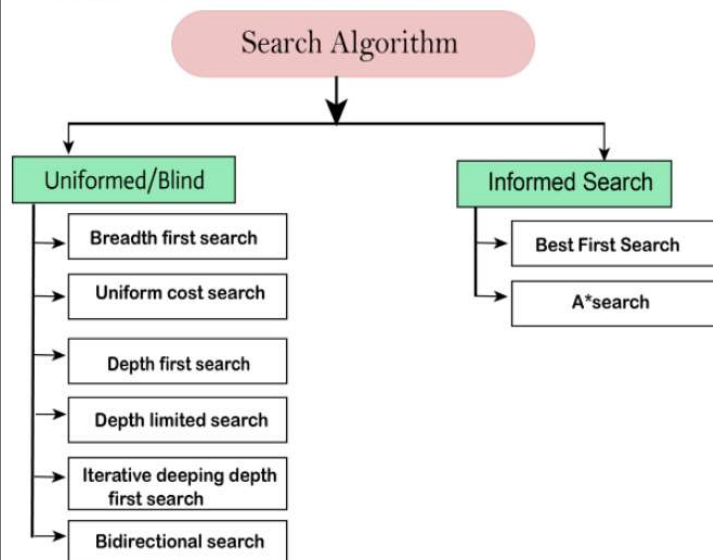


An informed search strategy is one that uses problem-specific knowledge beyond the definition of the problem itself.

- Informed search strategies use information about the problem (estimated distance from a state to the goal) to guide the search.
- An informed search can find solutions more efficiently than can an uninformed strategy.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



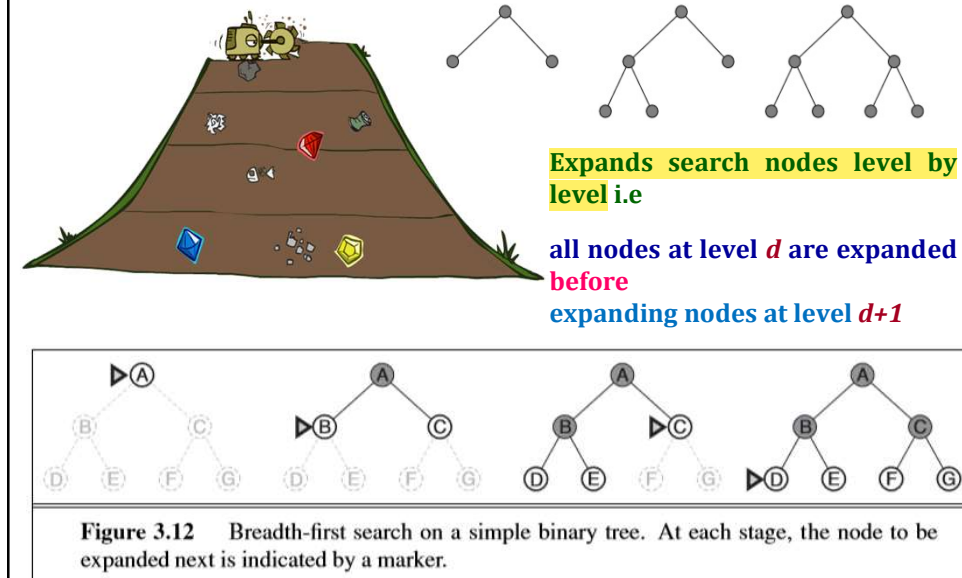
Breadth-First Search

Breadth-First Search

Breadth-first Search is a simple strategy in which

- the root node is expanded first
- then *expand a shallowest node first or shallowest unexpanded node* is chosen for expansion
- then all the successors of the root node are expanded next
- then their successors, and so on.
- fringe is a FIFO queue
- explore all possible branches at the same time
- move from level to level or expands search nodes level by level
- finding all possible branches at each level before moving on to the next level
- all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.

Breadth-First Search



Breadth-First Search

Breadth-first search is an instance of the general graph-search algorithm (Figure 3.7).

This is achieved very simply by using a FIFO queue for the frontier.

Thus,

new nodes (which are always deeper than their parents)
go to the back of the queue, and

old nodes, which are shallower than the new nodes,
get expanded first.

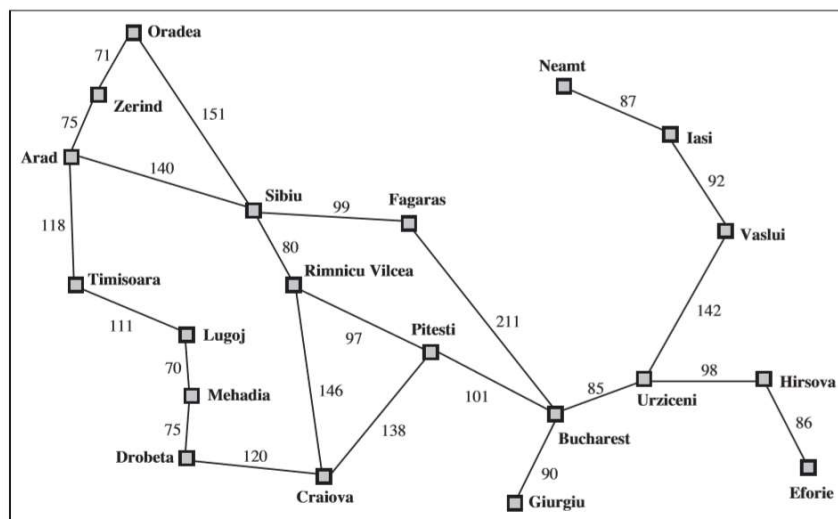
```

function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier) /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)

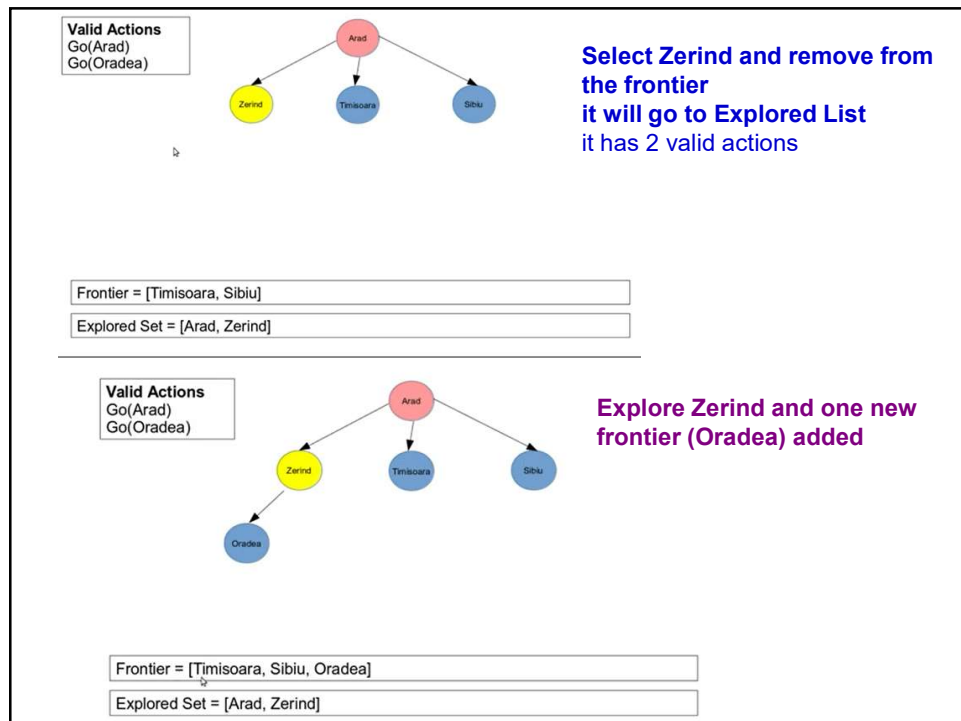
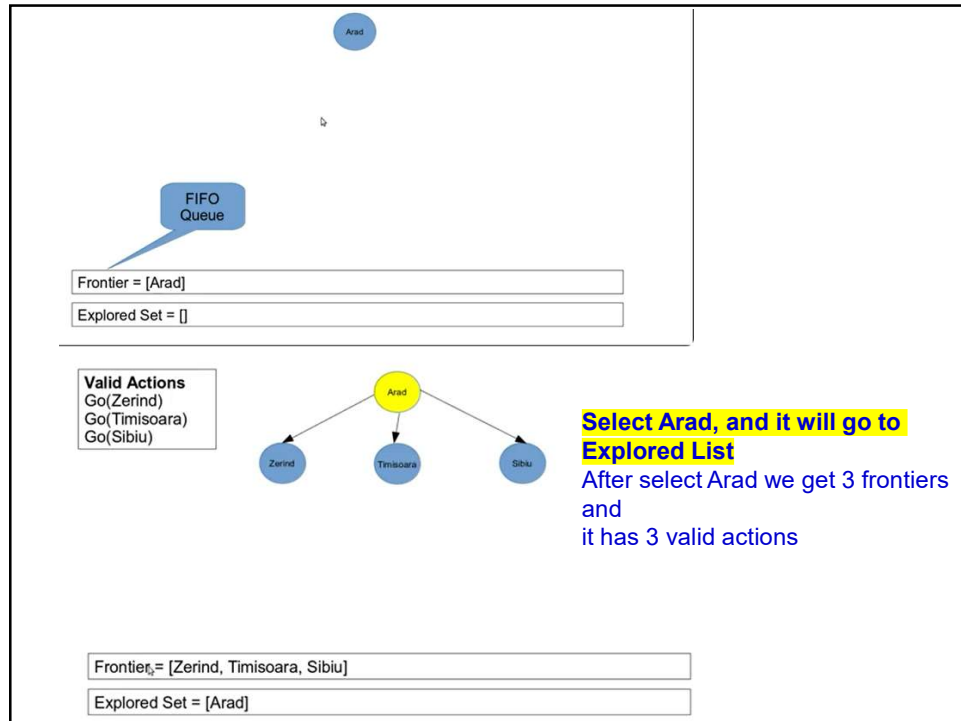
```

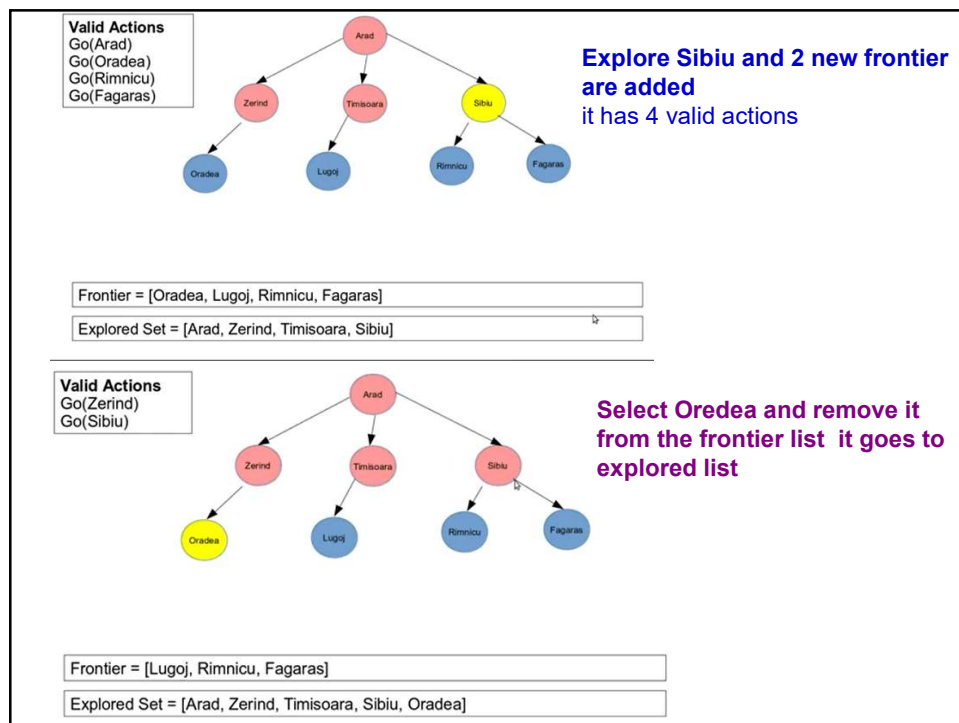
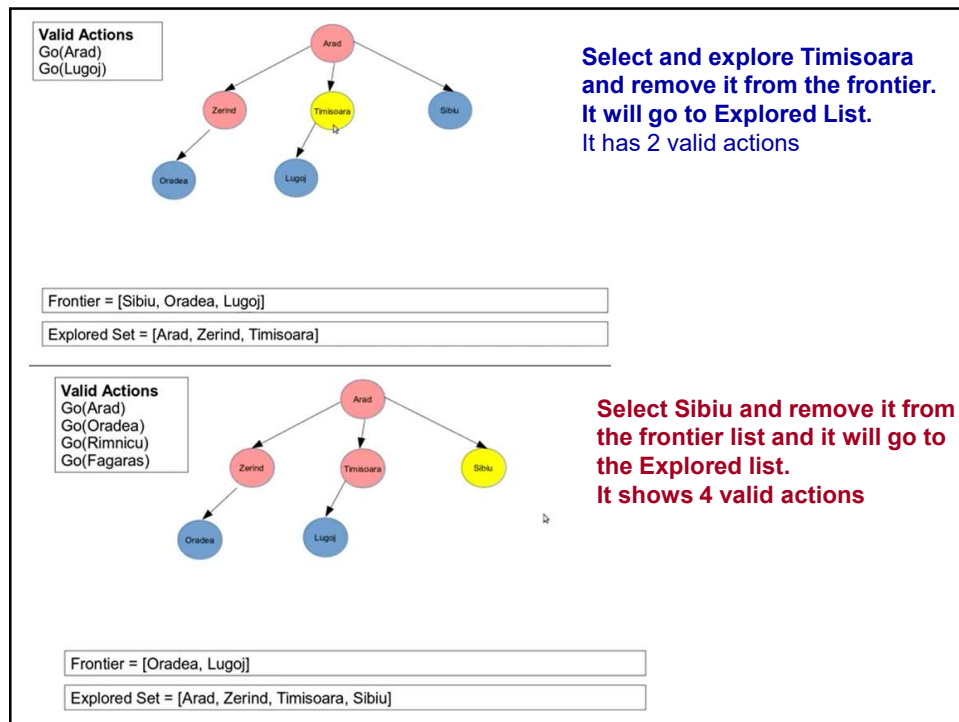
Figure 3.11 Breadth-first search on a graph.

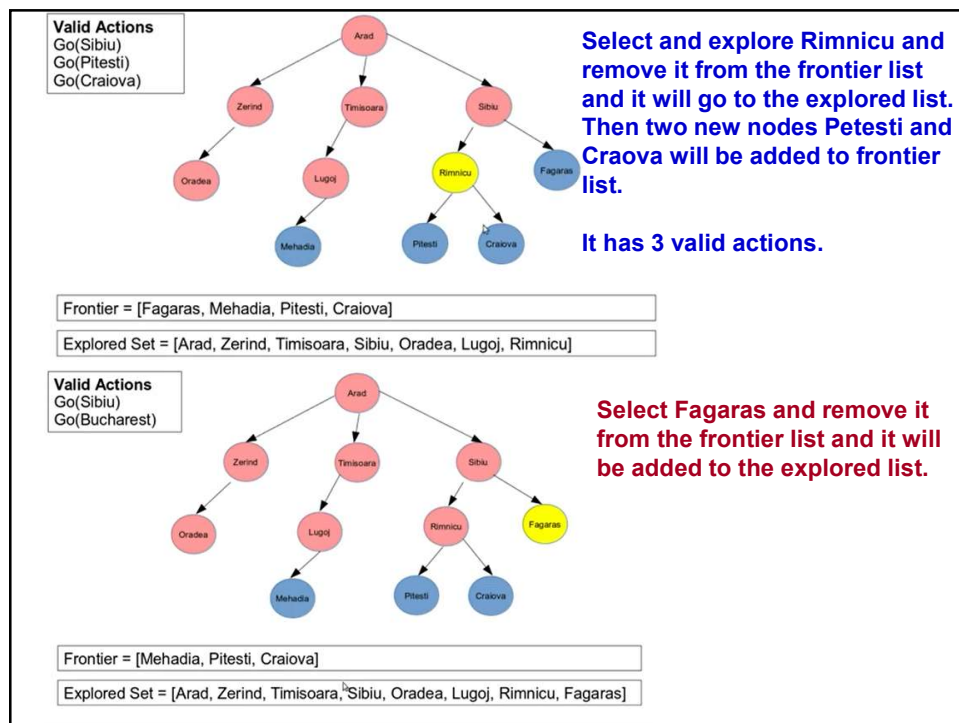
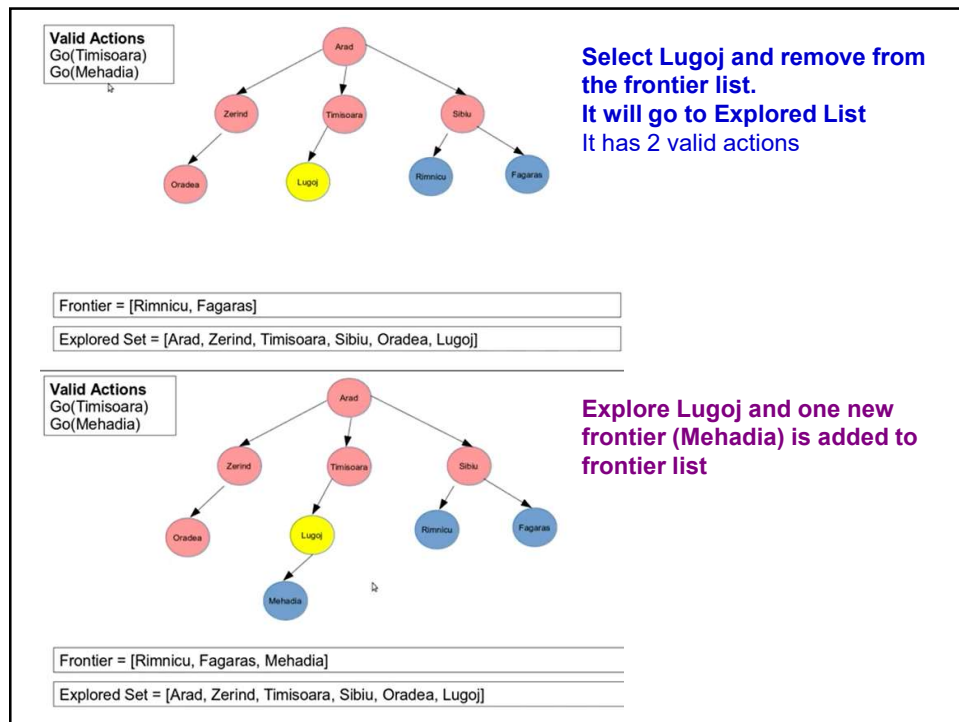
Breadth First Search

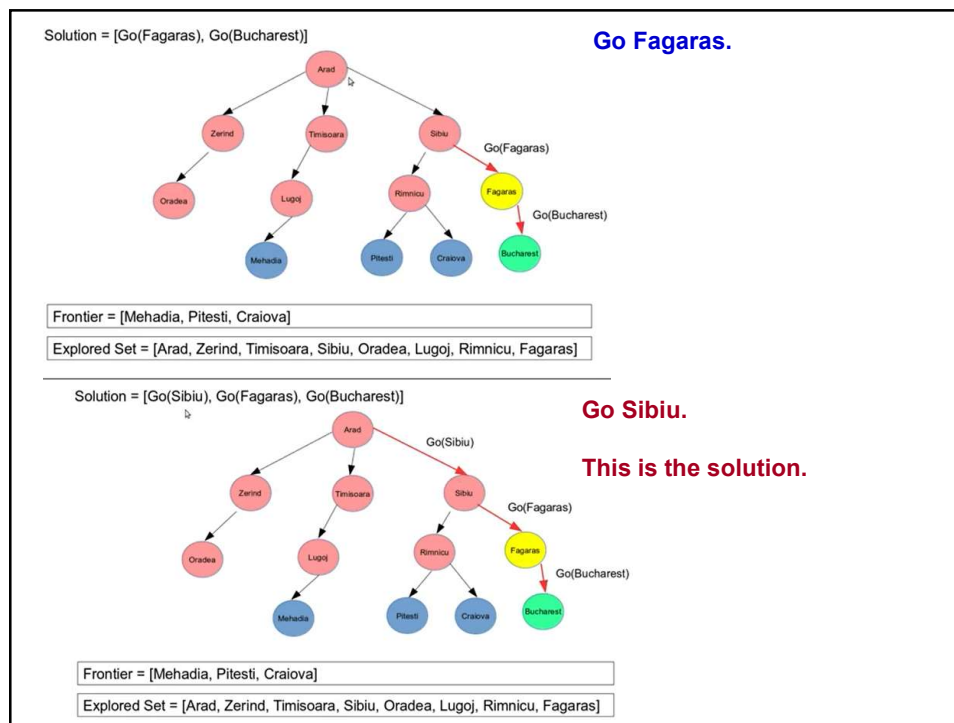
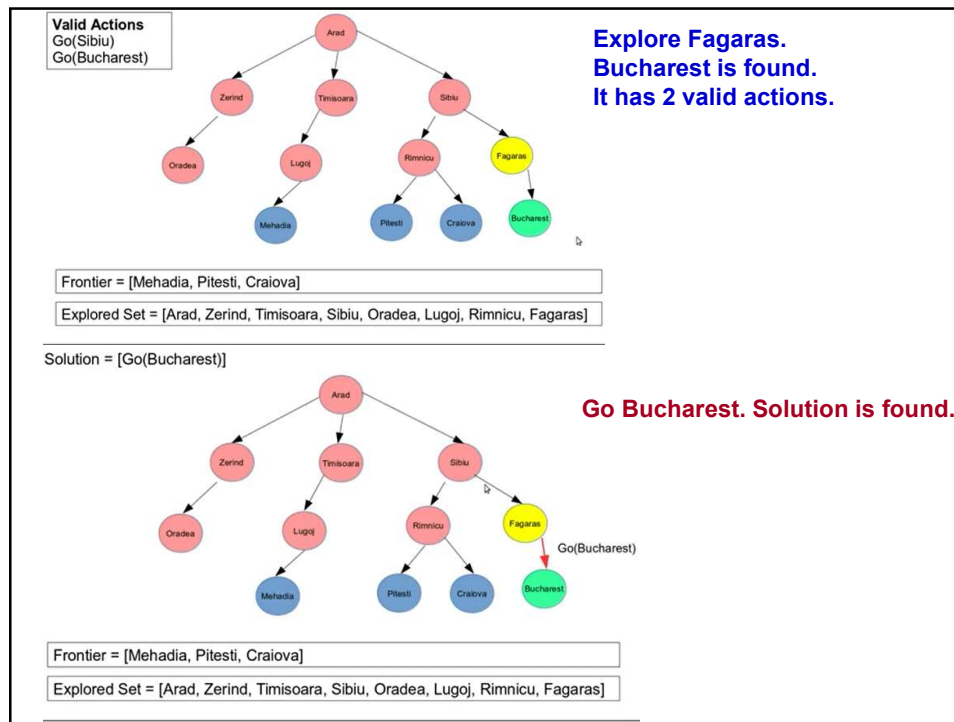


Prof. Dr. Md. Ashikur Rahman Khan









```

function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

```

```

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set

```

Figure 3.7 An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.

Breadth-first Search

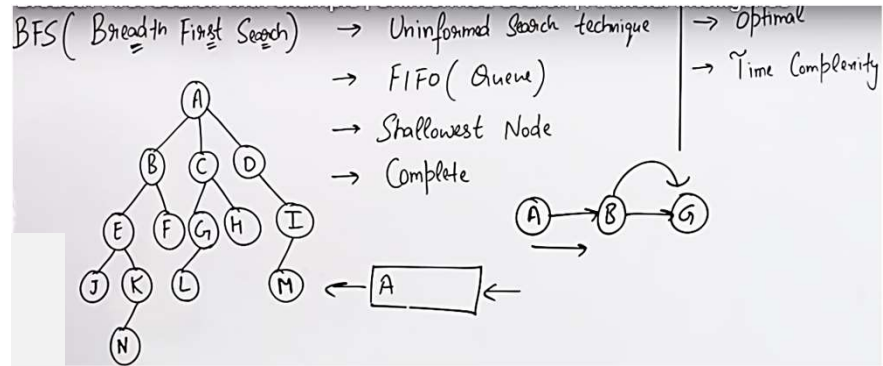
There is one slight tweak on the general graph-search algorithm, which is that **the goal test is applied to each node when it is generated rather than when it is selected for expansion.**

This decision is explained below, where we discuss time complexity.

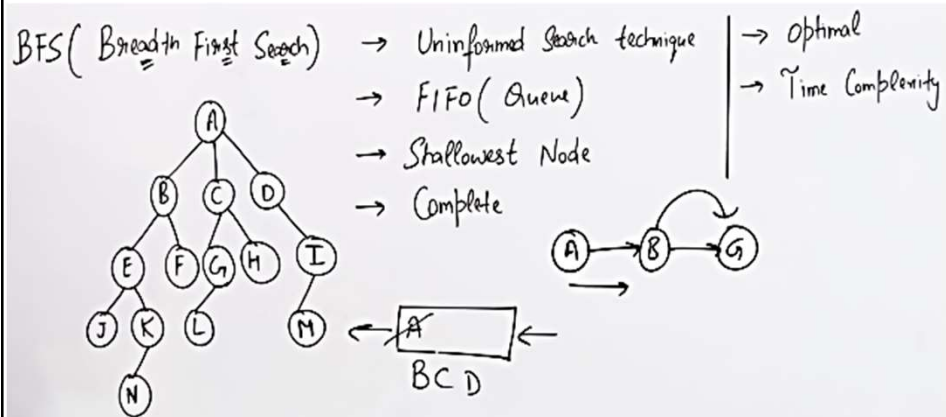
Note also that the algorithm, following the general template for graph search, discards any new path to a state already in the frontier or explored set; it is easy to see that any such path must be at least as deep as the one already found.

Thus, breadth-first search always has the shallowest path to every node on the frontier.

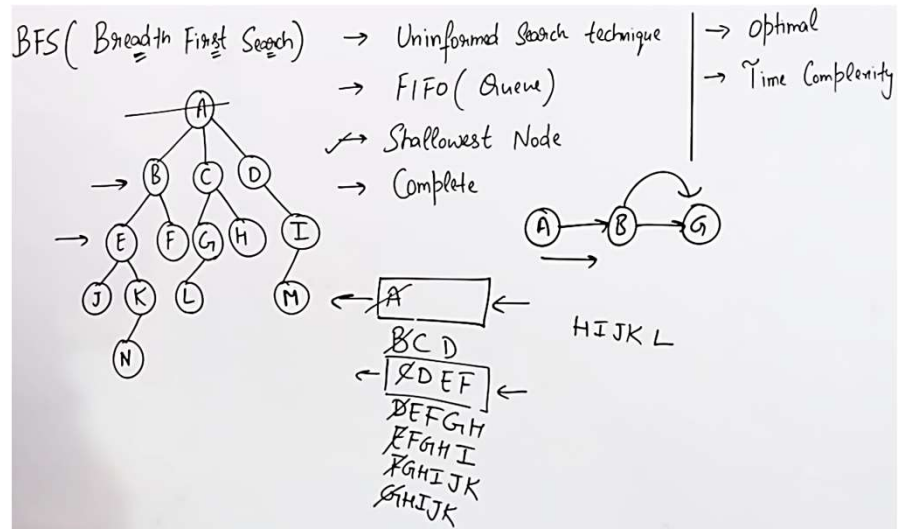
Breadth-first Search



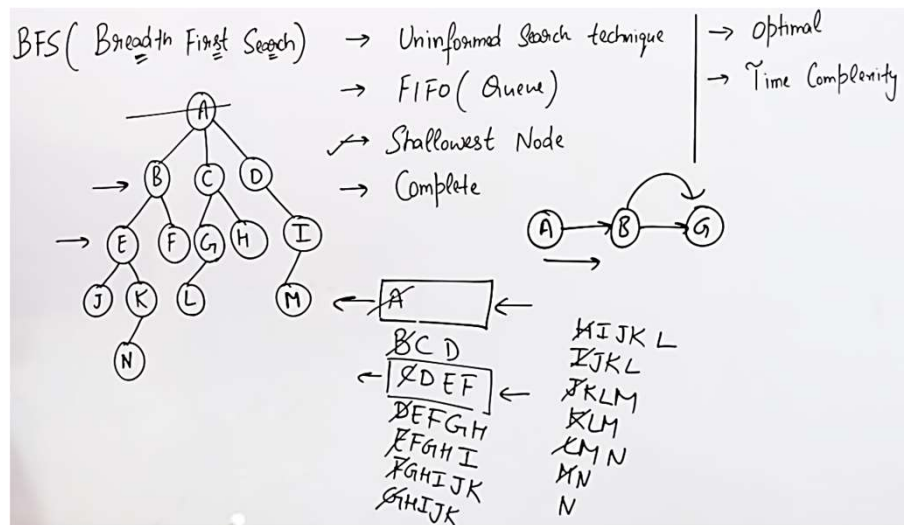
Breadth-First Search



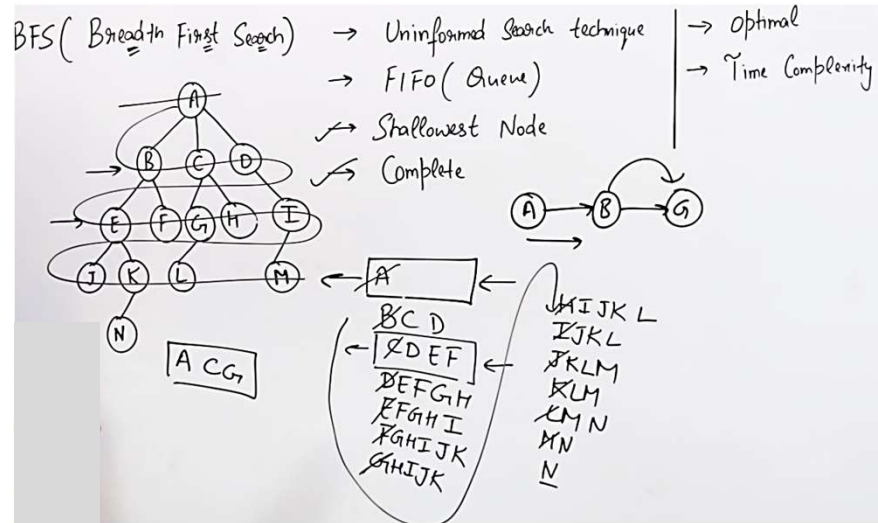
Breadth-First Search



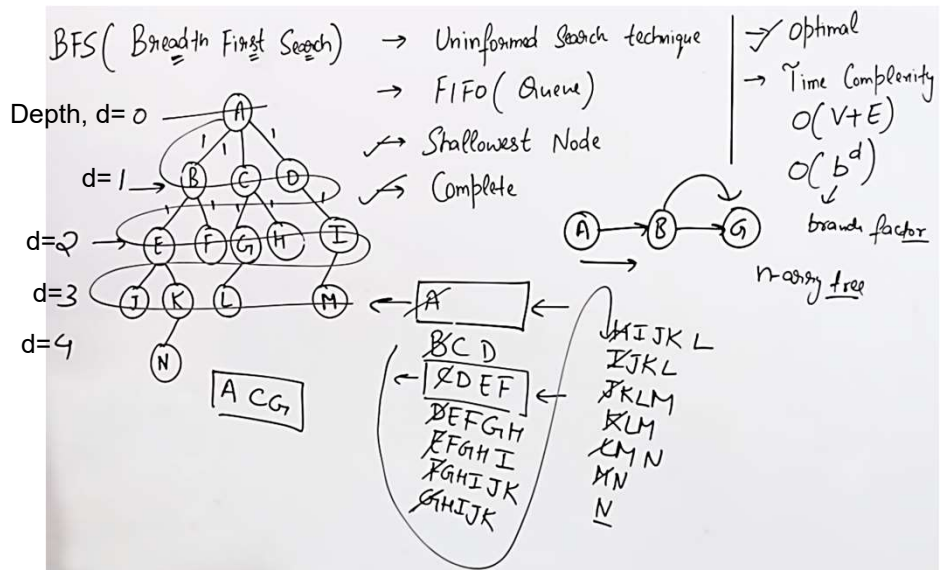
Breadth-First Search



Breadth-First Search



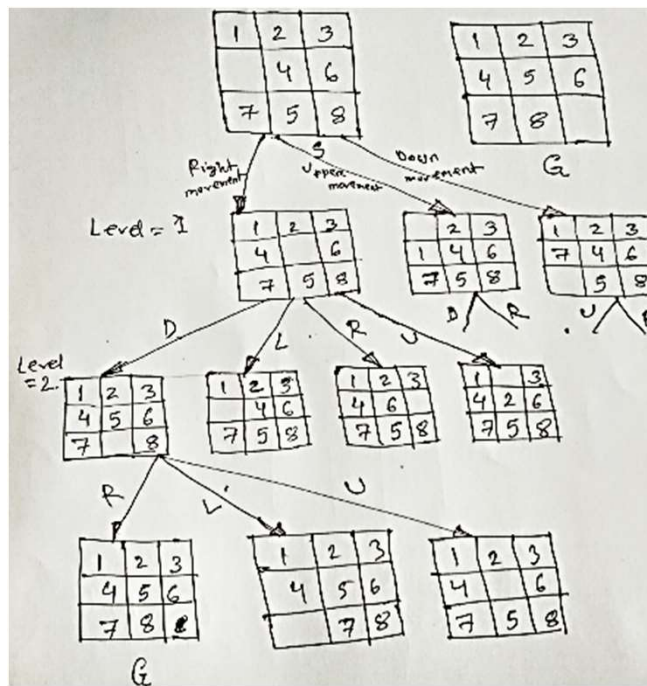
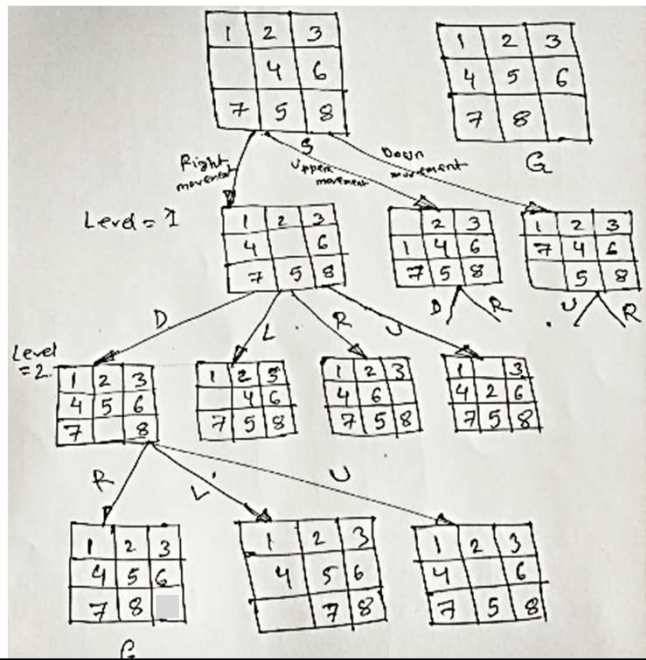
Breadth-First Search

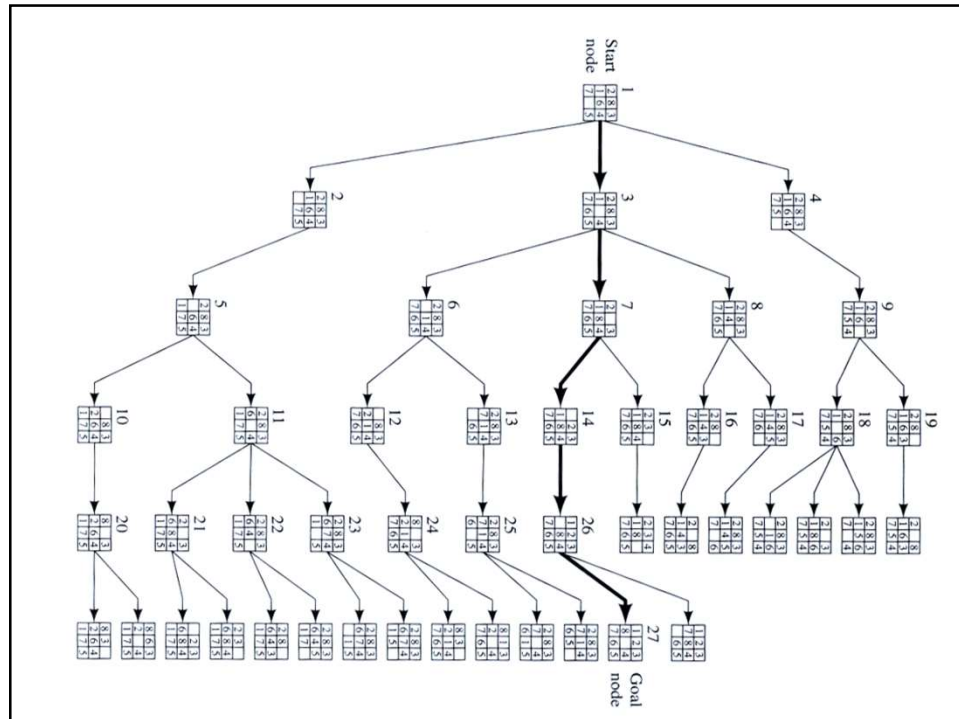


8-Puzzle problem without heuristic i.e (Breadth-First search)

7	2	4
5		6
8	3	1

Start State





Breadth-First Search

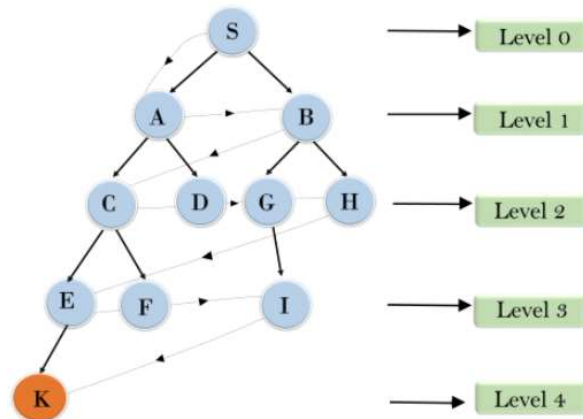
Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be: **(Consider each parent node possesses two child nodes only)**

S----> A---->B---->C---->D---->G---->H---->E---->F---->I---->K

Breadth-First Search

Breadth First Search

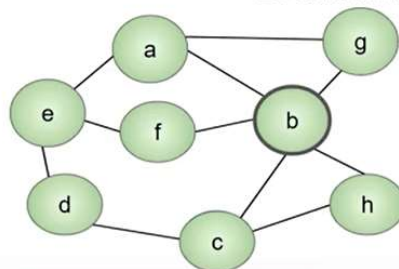


S---> A--->B--->C--->D--->G--->H--->E--->F--->I--->K

35

Breadth First Search for Graph Traversal

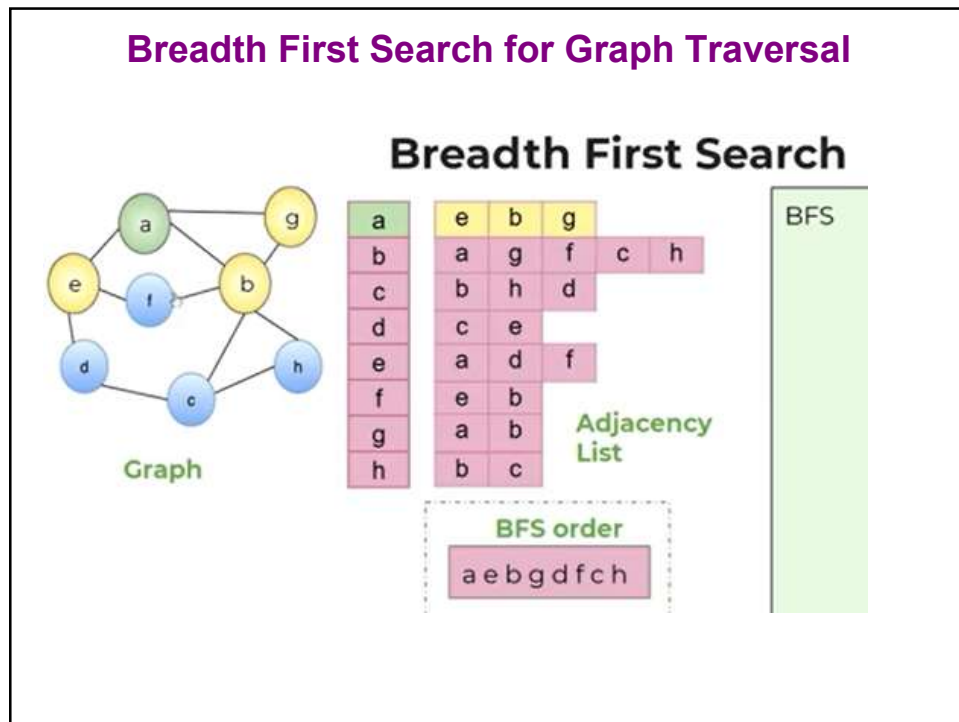
Breadth First Search



Select Node
↓
Explore completely

a e **b** g d f c h

Breadth First Search for Graph Traversal



Breadth-first Search

Pseudocode is given in Figure 3.11. Figure 3.12 shows the progress of the search on a simple binary tree.

```

function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier) /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
  
```

Figure 3.11 Breadth-first search on a graph.

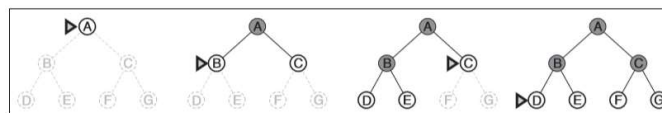


Figure 3.12 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

Performance Evaluation of Searching Algorithm

We can evaluate an algorithm's performance in four ways:

- 1) **Completeness:** Is the algorithm guaranteed to find a solution when there is one?
- 2) **Optimality:** Does the strategy find the optimal solution, as defined on page 68?
- 3) **Time complexity:** How long does it take to find a solution? Guaranteed to find the least cost path?

Time is often measured in terms of the number of nodes generated during the search.

- 4) **Space complexity:** How much memory is needed to perform the search?

Space in terms of the maximum number of nodes stored in memory.

Performance Evaluation or Measuring Performance of Search Algorithm

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Measuring Performance Of The Search Algorithms

Completeness

Is the algorithm guaranteed to find a solution when there is one?

Optimality

Does the strategy find the optimal solution?

Time complexity

How long does it take to find a solution?

Space complexity

How much memory is needed to perform the search?

Performance Evaluation or Measuring Performance of Search Algorithm

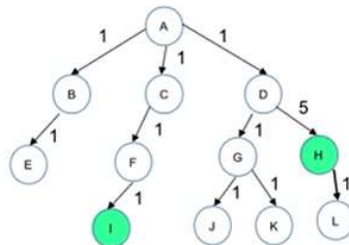
Measuring Performance Of The Search Algorithms

Completeness

Is the algorithm guaranteed to find a solution when there is one?

Optimality

Does the strategy find the optimal solution?



We have two goal nodes here

The two solutions are:

A → D → H with cost = 1 + 5 = 6

A → C → F → I with cost = 1 + 1 + 1 = 3

The solution A → C → F → I is the optimal solution

Performance Evaluation or Measuring Performance of Search Algorithm

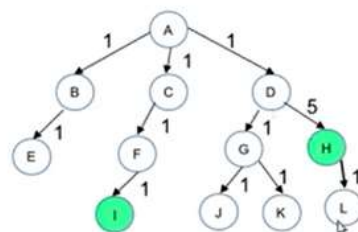
Measuring Performance Of The Search Algorithms

Notations:

b = **branching factor** = maximum number of successors of any node

d = the depth of the shallowest goal node

m = the maximum length of any path in the state space



Depth = 0

$b = 3$

Depth = 1

$d = 2$

Depth = 2

$m = 3$

Depth = 3

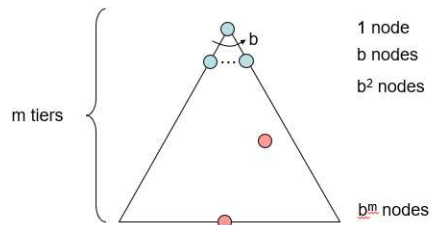
Performance Evaluation or Measuring Performance of Search Algorithm

Complexity is expressed in terms of three quantities:

- **b**, the branching factor or maximum number of successors of any node;
- **d**, the depth of the shallowest goal node
(i.e., the number of steps along the path from the root); and
- **m**, the maximum depth or length of any path in the state space.

- Number of nodes in entire tree?

$$1 + b + b^2 + \dots + b^m = O(b^{m+1})$$

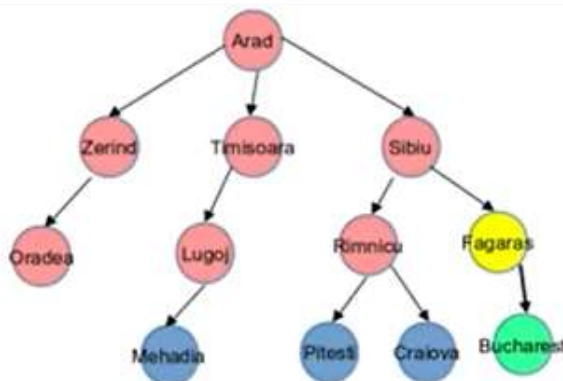


Performance Measurement of Breadth-first Search

1. Completeness: If the shallowest goal node is at some finite depth d , breadth-first search will eventually find it after generating all shallower nodes (provided the branching factor b is finite).

BFS is complete

if the shallowest goal node is at some finite depth d , breadth-first search will eventually find it after generating all shallower nodes



Measuring Performance of Breadth First Search Algorithm

2. Optimality:

Note that as soon as a goal node is generated,

it is the shallowest goal node because all shallower nodes must have been generated already and failed the goal test.

Now, the **shallowest goal node is not necessarily the optimal one.**

Breadth-first search is optimal if the path cost is a nondecreasing function of the depth of the node.

The most common such scenario is that all actions have the same cost.

Measuring Performance of Breadth First Search Algorithm

2. Optimality

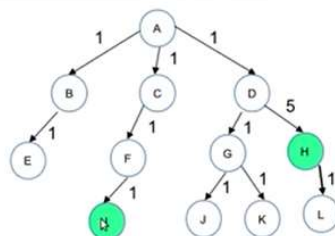
Example # 1:

Completeness

Is the algorithm guaranteed to find a solution when there is one?

Optimality

Does the strategy find the optimal solution?



We have two goal nodes here

The two solutions are:

A -> D -> H with cost = 1 + 5 = 6

A -> C -> F -> I with cost = 1 + 1 + 1 = 3

The solution A -> C -> F -> I is the optimal solution

Measuring Performance of Breadth First Search Algorithm

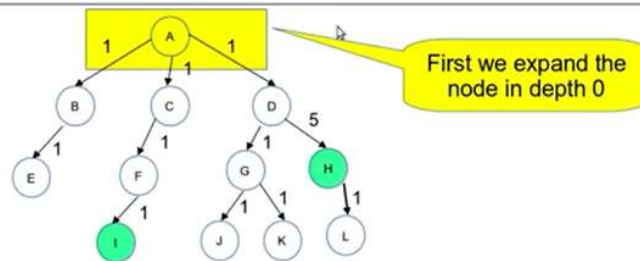
2. Optimality

Example # 1:

BFS is not optimal

BFS always returns the shallowest goal node

The shallowest goal node is not necessarily the optimal one



Measuring Performance of Breadth First Search Algorithm

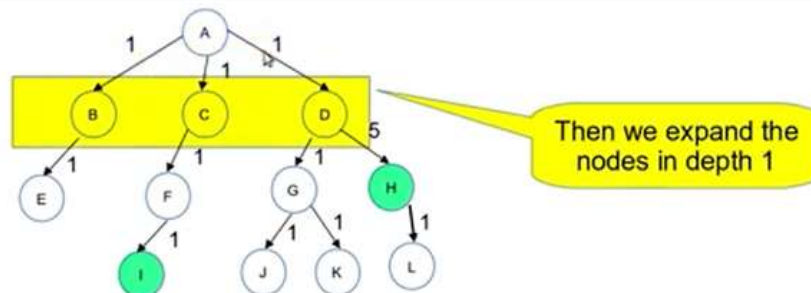
2. Optimality

Example # 1:

BFS is not optimal

BFS always returns the shallowest goal node

The shallowest goal node is not necessarily the optimal one



Measuring Performance of Breadth First Search Algorithm

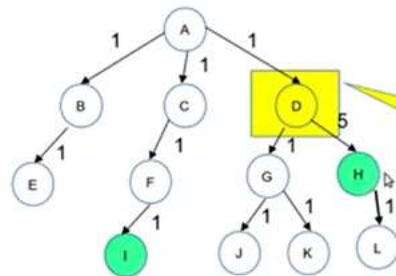
2. Optimality

Example # 1:

BFS is not optimal

BFS always returns the shallowest goal node

The shallowest goal node is not necessarily the optimal one



When we expand node D, we will generate the goal node H. So we will stop and return the solution A -> D -> H

Measuring Performance of Breadth First Search Algorithm

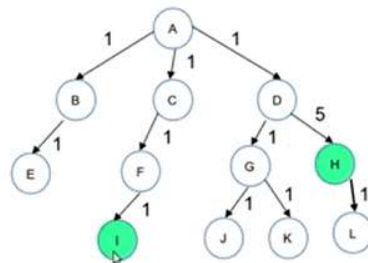
2. Optimality

Example # 1:

BFS is not optimal

BFS always returns the shallowest goal node

The shallowest goal node is not necessarily the optimal one



The solution A -> D -> H will be returned

The cost of this solution is: $1 + 5 = 6$

Measuring Performance of Breadth First Search Algorithm

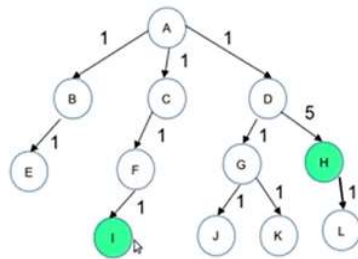
2. Optimality

Example # 1:

BFS is not optimal

BFS always returns the shallowest goal node

The shallowest goal node is not necessarily the optimal one



The solution A -> D -> H will be returned

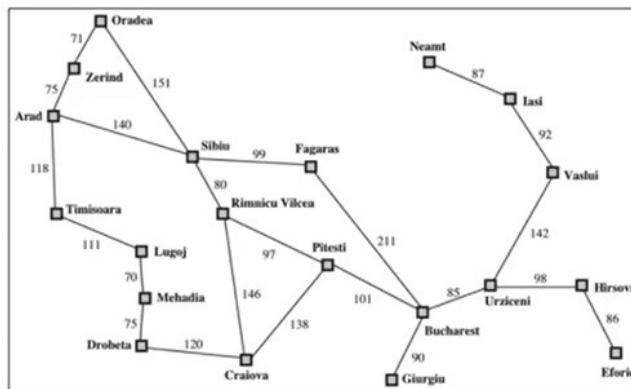
The cost of this solution is: $1 + 5 = 6$

However, a better solution exists:
A -> C -> F -> I, where the cost is $1 + 1 + 1 = 3$

Measuring Performance of Breadth First Search Algorithm

2. Optimality

Example # 2:



Solution returned was: Arad -> Sibiu -> Fagaras -> Bucharest
Path Cost = $140 + 99 + 211 = 450$

Another possible solution: Arad -> Sibiu -> Rimnicu -> Pitesti -> Bucharest
Path Cost: $140 + 80 + 97 + 101 = 418$

How BFS is not optimal? and When it is optimal? and Conditions of Optimality of BFS

How BFS is not optimal? and When it is optimal? and Conditions of Optimality of BFS

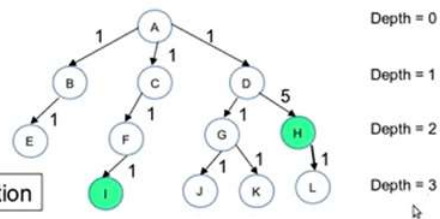
BFS do not always return the optimal solution

We have two goal nodes here

The two solutions are:
A -> D -> H with cost = 1 + 5 = 6
A -> C -> F -> I with cost = 1 + 1 + 1 = 3

The solution A -> C -> F -> I is the optimal solution

However, BFS will return the solution A -> D -> H as H is the shallowest goal node and BFS will reach it first.



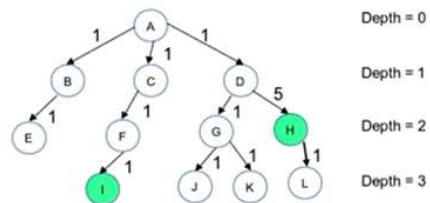
BFS do not always return the optimal solution

BFS do not return optimal solution when path cost is not a nondecreasing function of the depth of the node

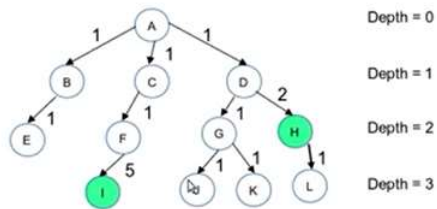
For node H, depth = 2, path cost = 6
For node I, depth = 3, path cost = 3

Thus a node with higher depth has path cost lower than some node in a shallower depth.

This means path cost is not a nondecreasing function of the depth of the node.

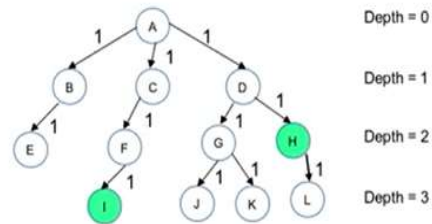


However, if path cost is a nondecreasing function of the depth of the node, BFS will return an optimal solution.



However, if path cost is a nondecreasing function of the depth of the node, BFS will return an optimal solution.

A special case of the above is when all the step costs are equal.

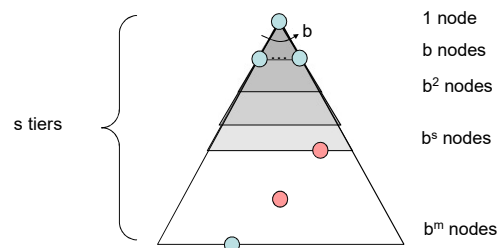


Breadth-first search is optimal if all step costs are equal, or if step costs is increasing with depth (e.g. constant). Because it always expands the shallowest unexpanded node.

Performance Measurement of Breadth-first Search

iii. Time complexity: So far, the news about breadth-first search has been **good. time and space is not so good.**
The news about

Consider a uniform tree where every state has b successors.



- The root of the search tree generates b nodes at the first level.
- Each of which generates b more nodes, for a total of b^2 at the second level.
- Each of these generates b more nodes, yielding b^3 nodes at the third level, and so on.

Measuring Performance of Breadth First Search Algorithm

3. Time Complexity

Time Complexity of BFS

Let us consider the branching factor to be 'b'.

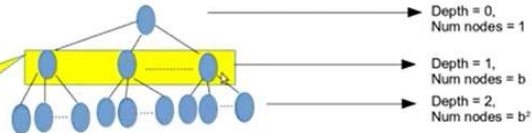
Let us assume that the shallowest goal node is at depth 'd'.

We will start expanding at root node at depth 0.



Time Complexity of BFS

For each of the 'b' nodes in depth 1, 'b' nodes will be generated. Thus in depth 2, there will be b^2 nodes

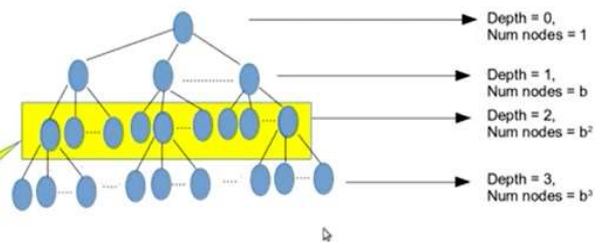


Measuring Performance of Breadth First Search Algorithm

3. Time Complexity

Time Complexity of BFS

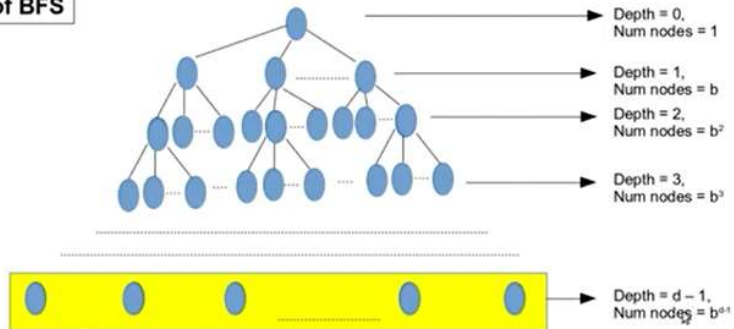
For each of the 'b' nodes in depth 2, 'b' nodes will be generated. Thus in depth 3, there will be b^3 nodes



Measuring Performance of Breadth First Search Algorithm

3. Time Complexity

Time Complexity of BFS



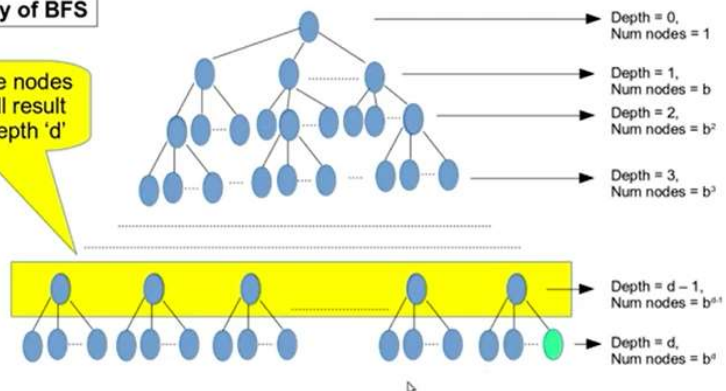
Continuing this way,
we will have b^{d-1} nodes
at depth $d-1$

Measuring Performance of Breadth First Search Algorithm

3. Time Complexity

Time Complexity of BFS

Expanding all the nodes
in depth ' $d-1$ ' will result
in b^d nodes at depth ' d '

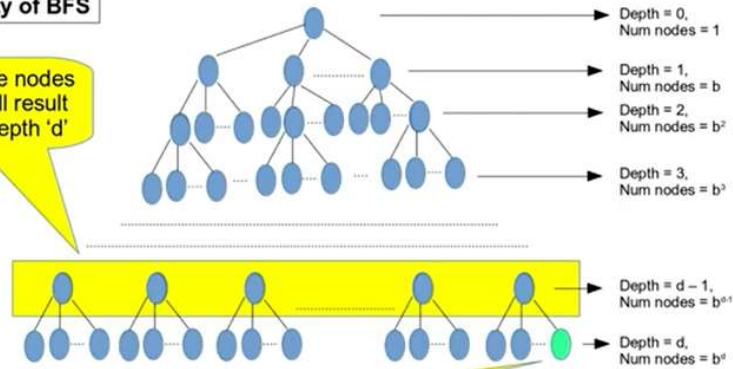


Measuring Performance of Breadth First Search Algorithm

3. Time Complexity

Time Complexity of BFS

Expanding all the nodes in depth 'd-1' will result in b^d nodes at depth 'd'

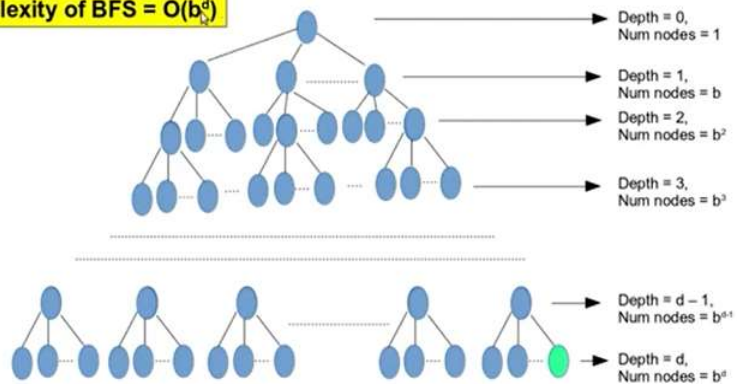


As the goal node gets generated, the search algorithm will stop

Measuring Performance of Breadth First Search Algorithm

3. Time Complexity

Thus, time complexity of BFS = $O(b^d)$



Total number of nodes generated = $1 + b + b^2 + b^3 + \dots + b^{d-1} + b^d = O(b^d)$

Measuring the performance of Breadth-first Search (Contd.)

iii. Time complexity (Contd.):

(i) When $b = 10$ and $d = 2$

Then the total number of nodes generated is

$$O(b^2) = b + b^2$$

$$\begin{aligned} O(b^d) &= 10 + 10^2 = 10 + 100 \\ &= 110 \end{aligned}$$

(ii) Similarly, When $b = 10$ and $d = 4$

Then the total number of nodes generated is

$$O(b^2) = b + b^2 + b^3 + b^4$$

$$\begin{aligned} O(b^4) &= 10 + 10^2 + 10^3 + 10^4 = 10 + 100 + 1000 + 10000 \\ &= 11,110 \end{aligned}$$

iii. Time complexity (Contd.): **Breadth-first Search**

Then the total number of nodes generated is

$$O(b) = b + b^2 + b^3 + \dots + b^d$$

Time required **1000 000 nodes/second** and suppose $b = 10$

Thus, Time required

Depth	Number of node(s)	Time Required
$d = 1$	$O(b) = b = 10$	$10/1000 = 0.01 \text{ ms}$
$d = 2$	$O(b) = b + \dots + b^d = b + b^2$ $= 10 + 10^2 = 10 + 100 = 110$	$110/1000 = 0.11 \text{ ms}$
$d = 4$	$O(b) = b + b^2 + b^3 + \dots + b^d =$ $= 10 + 10^2 + 10^3 + 10^4$ $= 10 + 100 + 1000 + 10000$ $= 11110$	$11110/1000 = 11$
$d = 6$	$O(b) = b + b^2 + b^3 + \dots + b^d =$ $= 10 + 10^2 + 10^3 + 10^4 + 10^5 + 10^6$ $= 10 + 100 + 1,000 + 10,000 + 100,000 + 1,000,000$ $= 1111110$	$1111110/1000$ $= 1111 \text{ ms}$ $= 1.1 \text{ s}$

Measuring the performance of Breadth-first Search (Contd.)

iii. Time complexity (Contd.):

Table (Continued)

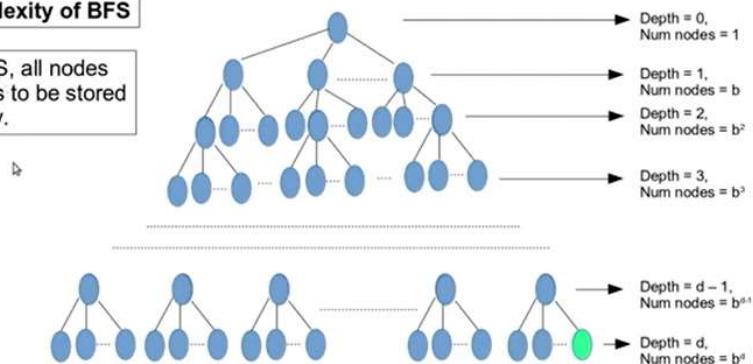
Depth	Number of node(s)	Time Required (ms)
d = 6	$O(b)=10^6$	$=1111110/1000 = 1111 \text{ ms}$ $= 1.1 \text{ s}$
d = 8	$O(b)=10^8$	$11111110/1000 = 11111.11 \text{ ms}$ $= 111.11111 \text{ s}$ $= 2 \text{ minutes}$
d = 10	$O(b)=10^{10}$	3 hours
d = 12	$O(b)=10^{12}$	13 days
d = 14	$O(b)=10^{14}$	3.5 years
d = 16	$O(b)=10^{16}$	350 years

Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

Space Complexity of BFS

In case of BFS, all nodes generated has to be stored in the memory.

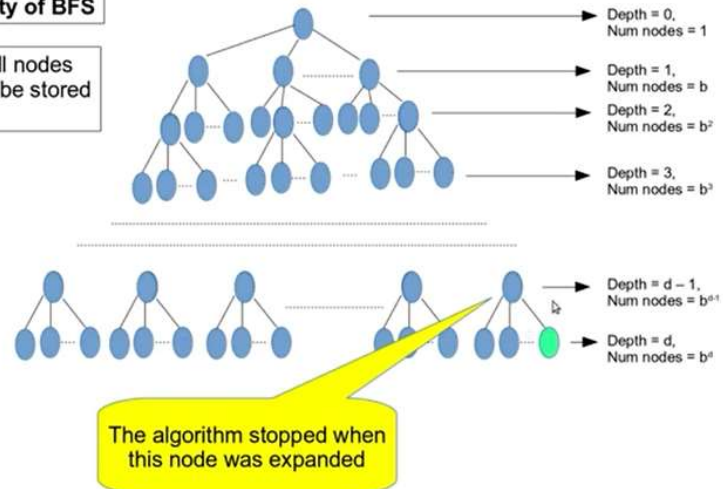


Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

Space Complexity of BFS

In case of BFS, all nodes generated has to be stored in the memory.

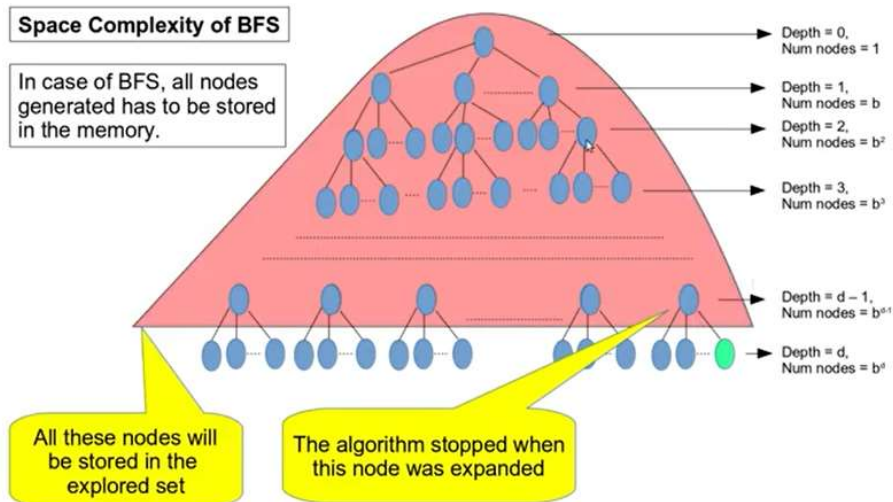


Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

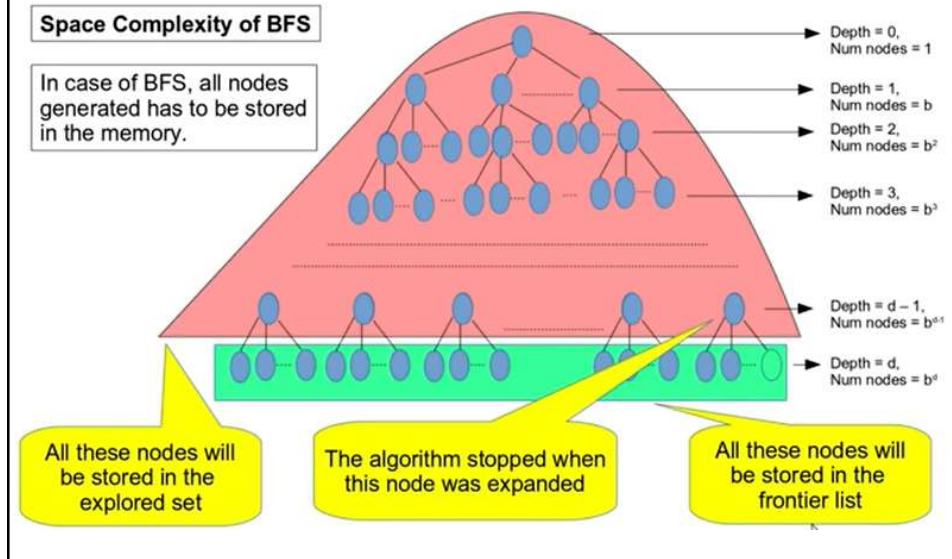
Space Complexity of BFS

In case of BFS, all nodes generated has to be stored in the memory.



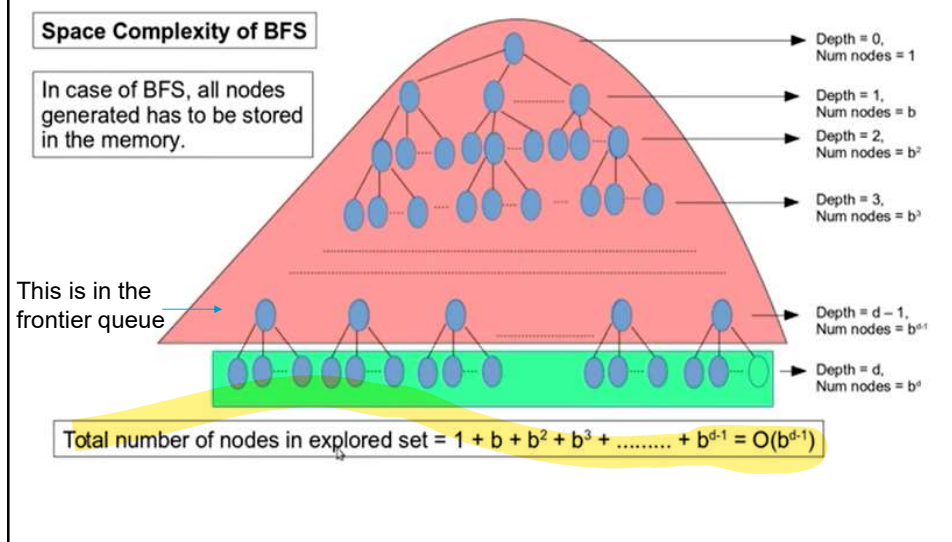
Measuring Performance of Breadth First Search Algorithm

4. Space Complexity



Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

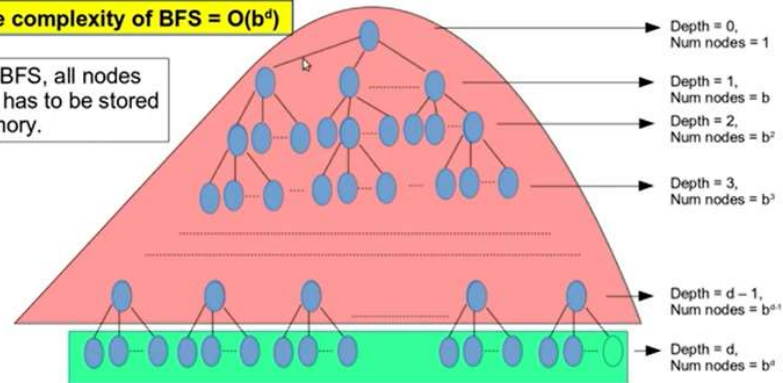


Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

Thus space complexity of BFS = $O(b^d)$

In case of BFS, all nodes generated has to be stored in the memory.



Total number of nodes in explored set = $1 + b + b^2 + b^3 + \dots + b^{d-1} = O(b^{d-1})$

Total number of nodes in the frontier list = $b^d = O(b^d)$

Total number of nodes in the memory = $1 + b + b^2 + b^3 + \dots + b^{d-1} + b^d = O(b^d)$

Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

Why is exponential complexity bound such as $O(b^d)$ bad?

Let us assume that 1 million nodes can be generated per second and that a node requires 1000 bytes of storage. Assume branching factor to be 10.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

Why is exponential complexity bound such as $O(b^d)$ bad?

Let us assume that 1 million nodes can be generated per second and that a node requires 1000 bytes of storage. Assume branching factor to be 10.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

The memory requirements are a bigger problem for breadth-first search than is the execution time

Measuring Performance of Breadth First Search Algorithm

4. Space Complexity

Why is exponential complexity bound such as $O(b^d)$ bad?

Let us assume that 1 million nodes can be generated per second and that a node requires 1000 bytes of storage. Assume branching factor to be 10.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

The memory requirements are a bigger problem for breadth-first search than is the execution time

Exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances

Measuring the performance of Breadth-first Search (Contd.)

iii. Space complexity:

Memory required 1000 bytes/node, Now suppose $b = 10$

Thus, Memory required

Depth	Number of node(s)	Memory Required (ms)
d = 1	$O(b) = b = 10$	= 10×1000 = 10,000 bytes = 10 kilo bytes
d = 2	$O(b) = 110$	= 110×1000 = 110,000 bytes = 110 kilobytes
d = 4	$O(b) = 11,110$	= 11110×1000 bytes = 11,110,000 bytes = 11.11 megabytes
d = 6	$O(b) = 1,111,110$	= $1,111,110 \times 1000$ bytes = 1,111,110,000 bytes = 1.111 gigabytes

Breadth-first Search

Then the total number of nodes generated is

Depth	Number of node(s)	Memory Required (ms)
d = 8	$O(b) = 10^8$	= $111,111,110 \times 1000$ bytes = 111,111,110,000 bytes = 111.111 gigabytes
d = 10	$O(b) = 10^{10}$	= $11111,111,110 \times 1000$ bytes = 11,111,111,110,000 bytes = 11.111 terabytes
d = 12	$O(b) = 10^{12}$	= $1111111,111,110 \times 1000$ bytes = 1,111,111,111,110,000 bytes = 1.111 petabytes
d = 14	$O(b) = 10^{14}$	= $111111111,111,110 \times 1000$ bytes = 111,111,111,111,110,000 bytes = 111.111 petabytes
d = 16	$O(b) = 10^{16}$	= $11111111111,111,110 \times 1000$ bytes = 11,111,111,111,111,110,000 bytes = 11.111 exabytes

Breadth-first Search

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

Assumes that

- (i) 1 million nodes can be generated per second and
- (ii) 1000 bytes of storage is required per node.

Many search problems fit roughly within these assumptions (give or take a factor of 100) when run on a modern personal computer.

Breadth-First Search

Advantages:

- BFS provides a solution if any solution exists.
- If there are more than one solutions for a given problem, then BFS provides the minimal solution which requires the least number of steps.

Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS need lots of time if the solution is far away from the root node.

Limitation of Breadth-first Search

Limitation/Disadvantages of breadth-first search:

From the above table it is found that

- (i) The memory requirements are a bigger problem for breadth-first search than is the execution time.

One might wait 13 days for the solution to an important problem with **search depth 12**

but no personal computer has the petabyte of memory it would take.

Fortunately, other strategies require less memory.

- (ii) Time is still a major factor. If the problem has a solution at depth 16, then (given our assumptions) it will take about **350 years** for breadth-first search (or indeed any uninformed search) to find it.

Limitation of Breadth-first Search

Breadth-First Search has one obvious disadvantage in that if it takes, say, a minimum of **d moves** to get from the Start state to the Goal state, and

if at each level, each state has an average (branch) of **b 'next' states**,

then before a path is finally found, we have generated approximately b^d states.

Such an exponential number can become very large very quickly, i.e. breadth-first search can take a lot of time.

In general, exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances.

